📅 DOSSIER DE CONCEPTION MENSUEL — IA-INSIGHT

SEPTEMBRE 2026 — MOIS 3 / 4

Facturation complète & Observabilité

Livrer le workflow de facturation end-to-end (CRA validé → facture envoyée → paiement suivi), et déployer la stack d'observabilité complète (métriques, logs, alertes, SLA).

📄 Facturation PDF/Excel | 💶 Suivi paiements | 📊 Prometheus/Grafana | 🔔 Alerting PagerDuty | 📈 SLA 99.5% | 🪵 Structured Logging

SOCIÉTÉ	SPRINTS	DURÉE	STATUT	ÉQUIPE
IA-INSIGHT	S5 + S6	4 semaines	📋 Planifié	3 devs + 1 DevOps

Sommaire

#	Section
1	Contexte & Besoin métier
2	Solution proposée & Périmètre
3	Architecture technique
4	Roadmap globale — 8 sprints / 4 mois
5	Sprints du mois — Septembre 2026
6	Sprint Review & Rétrospective

SECTION 1

Contexte & Besoin métier

Le problème des ESN aujourd'hui

Les ESN et cabinets de conseil IT gèrent des dizaines à des centaines de consultants disséminés chez leurs clients. Chaque mois, ce modèle génère une charge administrative considérable :

3h
PERDUES PAR CONSULTANT/MOIS EN ADMIN

23%
NOTES DE FRAIS AVEC ERREURS

8j
DÉLAI MOYEN FACTURATION POST-CRA

Processus	Situation actuelle	Impact
Notes de frais	Saisie manuelle Excel, scan PDF par email	Erreurs, pièces perdues, retards remboursement
CRA mensuel	Feuille Excel partagée ou formulaire papier	Retards validation → retards facturation client
Facturation client	Factures Word/Excel manuelles	Risques TVA, numérotation incohérente
Gestion congés	Email ou HR tool déconnecté des outils métier	Calcul frais km erroné si absences non remontées
Visibilité manager	Aucun tableau de bord temps réel	Arbitrages retardés, perte de contexte

Besoins consultant

- → Saisir une note de frais en langage naturel

Besoins manager / admin

- → Valider/refuser frais et CRA en 1 clic

- → Photographier un justificatif → OCR automatique
- → Frais km avec exclusion automatique absences/congés
- → Soumettre son CRA depuis n'importe quel appareil
- → Notifications temps réel sur les décisions admin
- → Génération automatique de factures PDF post-CRA
- → Tableau de bord global : frais, CRA, factures
- → Console multi-tenant : piloter plusieurs ESN
- → Conformité RGPD & audit trail LLM

SECTION 2

Solution proposée & Périmètre

> **Vision produit :** Plateforme SaaS multi-tenant d'automatisation administrative pour ESN. L'utilisateur parle à un assistant IA en français naturel — il s'occupe du reste.

✅ Dans le périmètre

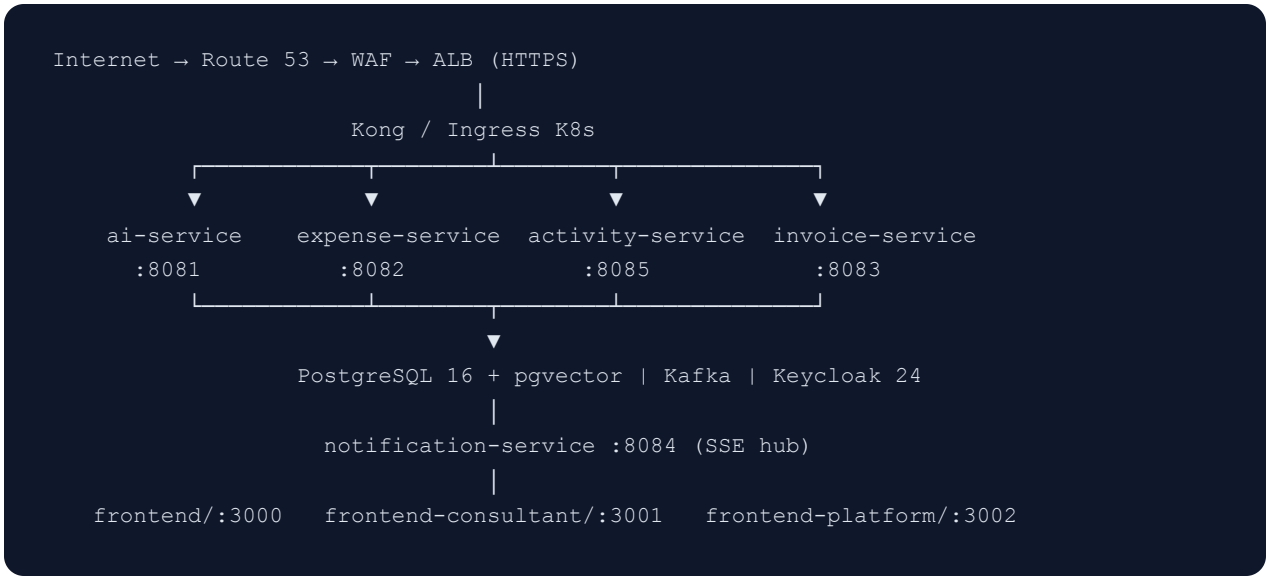
- ✓ Notes de frais — saisie libre, OCR, frais km
- ✓ CRA mensuel — workflow BROUILLON→SOUMIS→VALIDÉ
- ✓ Congés CP/RTT — demandes & solde
- ✓ Facturation — PDF & Excel client
- ✓ Notifications SSE temps réel
- ✓ Pipeline RAG pgvector + LLM Groq/OpenAI
- ✓ Multi-tenant Keycloak realm-per-tenant
- ✓ Gouvernance IA (audit, PII masking, token budget)
- ✓ Conformité RGPD (droit à l'oubli, portabilité)
- ✓ AWS Production (EKS, RDS, MSK, DR)

Stack technique

Couche	Technologie
LLM principal	Groq — llama-3.1-8b + gpt-oss-120b
LLM fallback	OpenAI gpt-4.1-mini (circuit breaker)
Embeddings	OpenAI text-embedding-3-large
DB + Vector	PostgreSQL 16 + pgvector
Auth	Keycloak 24 realm-per-tenant
Messaging	Apache Kafka KRaft
Backend	Spring Boot 3.3 / Java 21
Cloud	AWS EKS + RDS + MSK

SECTION 3

Architecture technique



Multi-tenant

Chaque organisation = 1 realm Keycloak. `TenantFilter` extrait le realm du JWT → `TenantContext` (ThreadLocal UUID). Toutes les requêtes JPA filtrent par `tenant_id`.

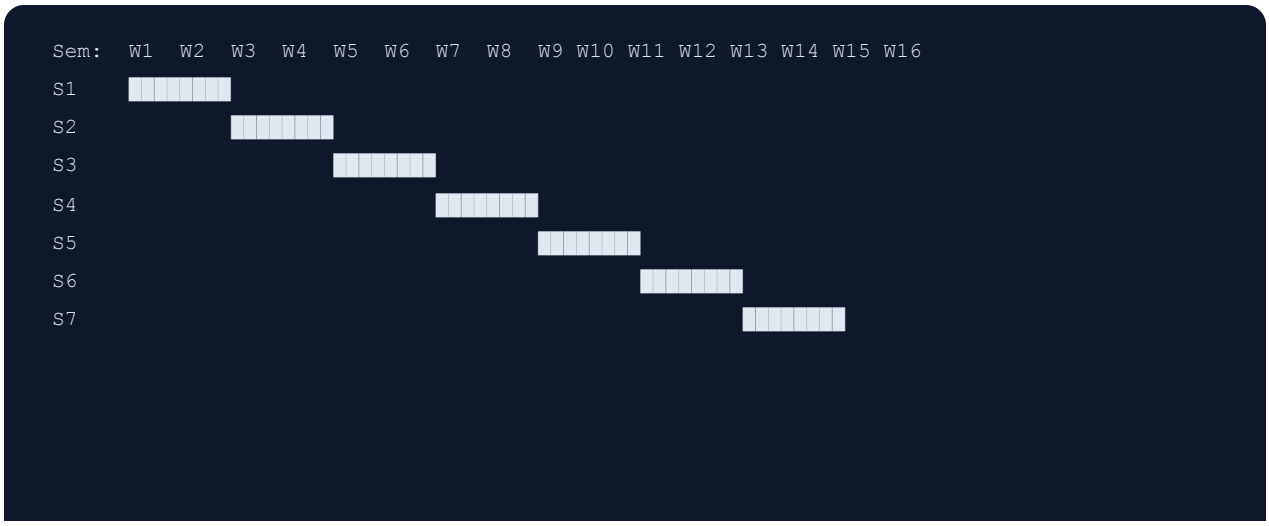
Résilience LLM

Groq primaire. Sur 5 échecs / 10 appels → circuit ouvert resilience4j → bascule immédiate OpenAI gpt-4.1-mini. Refermeture 30s. SSE broadcast via Kafka (group-id unique par pod `${HOSTNAME}`).

SECTION 4

Roadmap globale — 8 Sprints / 4 Mois

Sprint	Mois	Thème	Valeur livrée
S1	Juillet	Consolidation architecture + Kong	Architecture propre, routing vers chaque service
S2	Juillet	UX Design System & 3 frontends	Expérience unifiée, mobile WCAG AA
S3	Août	Gouvernance IA — Audit, PII Masking	0 PII vers LLM, traçabilité complète
S4	Août	Sécurité RGPD & Versioning modèles	DPIA validé, droit à l'oubli
S5	Septembre	Facturation end-to-end	Cycle CRA → Facture → Paiement
S6	Septembre	Observabilité & SLA	Grafana, alerting, runbooks
S7	Octobre	AWS Architecture & Terraform	EKS, RDS, MSK, Secrets Manager
S8	Octobre	Production Deployment & Go-Live	AWS prod, client pilote, SLA 99.9%




S8
Juillet
Août
Septembre
Octobre

CONTEXTE DU MOIS

Maturation produit & visibilité opérationnelle

Après la sécurisation de la gouvernance IA (Août), Septembre est le mois de la **maturité produit** : le cycle de facturation est complet et l'équipe dispose d'une visibilité totale sur la santé du système en production. C'est la condition sine qua non avant d'aborder la migration AWS en Octobre.

8j	0	<1h
DÉLAI MOYEN FACTURATION ACTUEL	DASHBOARDS DE MONITORING	MTTD CIBLE (DÉTECTION INCIDENTS)

S5

Facturation complète — Workflow CRA → Facture → Paiement

Semaines 9-10 · Septembre 2026 · Responsable : Tech Lead Backend

Workflow cible end-to-end

Étape	Déclencheur	Action	Service
1. CRA validé	Admin clique "Valider CRA"	Statut → VALIDÉ, event cra.validated publié	activity-service
2. Suggestion facture	Notification SSE à l'admin	Bouton "Générer la facture" apparaît dans le dashboard	frontend/
3. Génération	Admin clique "Générer"	PDF + Excel créés, facture en statut BROUILLON	invoice-service
4. Validation facture	Admin vérifie les données	Compléter clientCompanyName, unitPriceHt si manquants	frontend/
5. Envoi	Admin clique "Marquer Envoyée"	Statut → ENVOYÉE, date d'envoi loggée	invoice-service
6. Paiement	Admin reçoit le virement	Statut → PAYÉE, date de paiement + montant reçu	invoice-service
7. Relance	Scheduler J+30 sans paiement	Notification admin : relancer le client	invoice-service

Règles de calcul des dates (BUSINESS-RULES)

CRA period	invoiceDate	paymentDueDate (45j)
Juin 2026	2026-07-01	2026-08-15
Juillet 2026	2026-08-01	2026-09-15
Août 2026	2026-09-01	2026-10-16 (sam → lun)

User stories Sprint 5

- → En tant qu'admin, après validation d'un CRA, je vois une notification "Générer la facture pour {consultant} — {mois}" dans mon dashboard
- → En tant qu'admin, je génère la facture en 1 clic — les données seller sont pré-remplies depuis le profil, je complète uniquement le client et le TJM si absents
- → En tant qu'admin, je marque une facture comme "Envoyée" et la date d'envoi est enregistrée automatiquement
- → En tant qu'admin, je marque une facture comme "Payée" avec le montant reçu — l'écart par rapport au montant facturé est calculé
- → En tant qu'admin, je reçois une alerte 30 jours après l'envoi si la facture n'est pas encore payée
- → En tant que platform_admin, je consulte le chiffre d'affaires total par tenant et par mois

Livrables techniques

- ✓ InvoiceScheduler.java : job @Scheduled quotidien vérifiant les factures non payées à J+30
- ✓ Invoice statuts : BROUILLON → ENVOYÉE → PAYÉE | ANNULÉE
- ✓ Endpoint PATCH /invoices/{id}/mark-sent (date_envoi)
- ✓ Endpoint PATCH /invoices/{id}/mark-paid (montant_reçu, date_paiement)
- ✓ Dashboard admin : liste factures par statut + indicateurs CA mensuel
- ✓ Notification SSE "Facture en attente de génération" post-validation CRA
- ✓ Export comptabilité : CSV factures par période (pour comptable externe)

Refonte notes de frais : upload seul + aperçu éditable, traçabilité et anti-doublon

Livré en avance sur planning (implémenté pendant le sprint). Avis MOA : la saisie de texte factuel en parallèle du justificatif était redondante avec l'OCR/Vision et causait des rejets frustrants sur de simples différences de formulation. Nouveau flux : upload du justificatif → extraction automatique (type/montant/date/enseigne/ville) → **aperçu éditable** pour relecture/correction → confirmation. Un champ "Motif/contexte" optionnel est conservé pour la justification métier (jamais comparée au document). Le cross-check texte/document (`ExpenseMismatchDetector`) est supprimé — devenu obsolète sans texte factuel à comparer.

Backend

✓	Upload en deux étapes :	`POST /receipts/upload`	(OCR + Vision + extraction LLM, sans persistance) puis	`POST /receipts/confirm`	(persistance des champs, éventuellement édités dans l'aperçu)
✓	`ReceiptFileValidator` : format PDF/JPG/PNG vérifié par signature binaire (magic bytes), pas par extension déclarée — rejette un exécutable renommé				
✓	`ReceiptStorageService.findById()` : relocalise le fichier uploadé entre l'étape d'extraction et celle de confirmation				
✓	Détection de doublon exécutée aux	**deux**	étapes (extraction ET confirmation) — bloque dès l'aperçu, et re-vérifie après édition		
✓	Suppression de	`ExpenseMismatchDetector`	(+ tests) et de l'exigence de conformité stricte du texte saisi — obsolète, plus de texte factuel à comparer		
✓	Classification du type gérée entièrement par le prompt LLM (taxonomie fixe, multilingue) — aucune règle de catégorisation en dur côté code				
✓	Fix fallback Vision : conversion PDF→PNG correcte avant envoi à l'API Vision (bug bloquant tout PDF halluciné en "format image non supporté")				
✓	Endpoint `GET /expenses/{id}/receipt` : sert le justificatif stocké, réservé admin/manager				

F

✓

✓

- ✓ `DuplicateExpenseMatcher` : détection de doublons basée DB (même consultant/jour/type, montant ±1%, description similaire) — inchangé, réutilisé aux deux étapes du nouveau flux

Definition of Done — Sprint 5

- ✓ Workflow complet testé : CRA validé → facture générée → envoyée → payée (parcours E2E)
- ✓ invoiceDate = 1er du mois suivant le CRA (test unitaire)
- ✓ paymentDueDate = invoiceDate + 45j avec gestion week-end (test unitaire)
- ✓ Scheduler relance : déclenché en test avec délai artificiel, alerte SSE reçue
- ✓ Export CSV comptabilité : format validé par un comptable (test d'acceptation)
- ✓ Upload justificatif → aperçu éditable → confirmation — livré et testé (build vert ai-service + expense-service)
- ✓ Aperçu justificatif accessible depuis la console admin lors de la validation d'une note de frais — livré
- ✓ Enseigne et ville affichées distinctement (admin + consultant) — livré
- ✓ Détection de doublons bloquante (extraction + confirmation), basée DB — livrée et testée (unit tests DuplicateExpenseMatcher)

S6 Observabilité, SLA & Performance

Semaines 11-12 · Septembre 2026 · Responsable : DevOps + Tech Lead

Stack d'observabilité cible

Composant	Outil	Rôle	Déploiement
Métriques	Prometheus + Micrometer	Collecte métriques JVM, HTTP, Kafka, LLM	namespace monitoring (K8s)
Dashboards	Grafana	Visualisation temps réel + alertes visuelles	namespace monitoring
Logs structurés	Loki + Logback JSON	Centralisation logs JSON de tous les pods	namespace monitoring
Alerting	Alertmanager	Routage alertes → email/Slack/PagerDuty	namespace monitoring
Tracing	OpenTelemetry (partiel)	Traces distribuées inter-services	Side-car dans multi-agent
Uptime	Probe Kubernetes	Liveness + Readiness probes	Déjà en place — à enrichir

SLA cibles à définir et monitorer

Service Level Objectives (SLO)

Disponibilité globale plateforme	**≥ 99.5% / mois (3.6h downtime max)**
Latence p95 API (hors LLM)	**≤ 300ms**
Latence p95 appels LLM Groq	**≤ 3s (Groq normal)**
Latence p95 appels LLM fallback OpenAI	**≤ 8s**

Délai notification SSE (CRA validé → reçu)	**≤ 2s (p95)**
Taux d'erreur HTTP 5xx	**≤ 0.1%**
Taux succès extraction LLM (JSON valide)	**≥ 95%**
MTTD (Mean Time To Detect)	**≤ 1h**
MTTR (Mean Time To Recover)	**≤ 4h**

Dashboards Grafana à créer

- ✓ **Platform Overview** : SLA uptime, req/s, erreurs 5xx, CPU/RAM par pod
- ✓ **LLM Intelligence** : latence par modèle, tokens/min, coût estimé, taux fallback, taux d'erreur JSON
- ✓ **Business KPIs** : notes de frais créées/jour, CRA soumis/validés, factures générées/payées
- ✓ **Kafka Health** : lag consumer, throughput, partition distribution

- → Alertes Prometheus : pod restart > 3/h → CRITICAL, latence p95 > 1s → WARNING
- → Log format JSON unifié : timestamp, level, service, tenant_id, trace_id, message
- → Corrélation logs → traces → métriques (Grafana Explore)
- → Retention : métriques 30j, logs 15j, traces 7j

Serveur MCP — ouverture aux systèmes externes

Livré en avance sur planning (implémenté pendant le sprint) — un serveur MCP (Model Context Protocol) en lecture seule expose les notes de frais et factures d'un tenant à des systèmes externes (agents IA, outils partenaires), sans développement d'intégration sur mesure à chaque nouveau cas d'usage.

Backend

- ✓ Serveur MCP (transport SSE) intégré à ai-service — aucun microservice supplémentair l'empreinte ressources sur le cluster
- ✓ 4 outils `list_expenses` , `get_expense_summary` , `list_invoices` , `get_invoi`

en lecture seule :

- ✓ `InvoiceEntity` / `InvoiceJpaRepository` (nouveau, lecture seule) : accès direct à la table `invoice` partagée, tenant évite un appel ré service et le gap tenant qui exista serveur-à-serveu
- ✓ `McpApiKeyFilter` : authentification par clé API dédiée par tenant, pose le même `TenantContext` que le J isolatio
- ✓ Rate limiting (30 req/min par défaut) + journalisation de chaque appel d'outil pour l'au
- ✓ Spring AI MCP upgradé en 1.1.8 (depuis 1.0.0) : API de session officielle (`McpSyncServerExchange.sessionId()`) — simplifie la réso threads Reactor et à la réflexion sur le SDK

User stories Sprint 6

- → En tant que DevOps, je reçois une alerte Slack en moins de 5 minutes si un pod crashe ou si la latence LLM dépasse 5s
- → En tant que Tech Lead, je consulte sur Grafana la latence p95 LLM des 7 derniers jours ventilée par modèle (Groq vs OpenAI fallback)
- → En tant que platform_admin, je consulte le nombre de notes de frais créées par tenant sur le mois courant
- → En tant que DevOps, tous les logs des 5 microservices sont centralisés dans Loki avec trace_id permettant de reconstituer une requête cross-service
- → En tant que Tech Lead, je dispose d'un runbook pour chaque type d'alerte critique (pod down, DB connexions saturées, circuit breaker ouvert)

Definition of Done — Sprint 6

- ✓ Prometheus scrape toutes les métriques /actuator/prometheus des 5 services
- ✓ 4 dashboards Grafana opérationnels et importés via ConfigMap
- ✓ Test alerte : tuer un pod → alerte Slack reçue en < 5 min
- ✓ Logs JSON de tous les services centralisés dans Loki
- ✓ SLO Error Budget calculé et visible sur le dashboard Platform Overview
- ✓ Runbook pour 5 alertes critiques documenté dans docs/RUNBOOK-OPS.md
- ✓ Serveur MCP (lecture seule notes de frais + factures) livré et testé — isolation tenant vérifiée par clé API, build vert (unit tests McpApiKeyFilter + ExpenseInvoiceMcpTools)
- ✓ Validation avec un client MCP tiers (MCP Inspector) sur le transport SSE : connexion, liste des 4 outils, appels réels avec clé API valide/invalide, isolation cross-tenant testée
- ✓ Documentation d'intégration MCP livrée (docs/MCP-Integration-Guide.pdf) et présentation commerciale dédiée (docs/PITCH-MCP.pdf)

FIN DE MOIS

Sprint Review & Rétrospective Septembre

Démos à préparer

- → Parcours complet : valider CRA → générer facture → marquer envoyée → marquer payée
- → Dashboard Grafana LLM : montrer une période avec fallback Groq→OpenAI
- → Simuler un crash pod → montrer l'alerte Slack reçue
- → Rechercher une requête dans Loki avec son trace_id

Go / No-Go AWS (Octobre)

Critères Go pour Octobre :

✓ Workflow facturation complet livré
✓ SLO définis et mesurables
✓ Observabilité opérationnelle
✓ DPIA validé (livré en Août)
✓ 0 CVE critique dans OWASP scan